

Client Update

An AI-Driven Spatial–Temporal Framework for
Occlusion-Robust Sport Pose Estimation

An end-to-end technical walkthrough:

camera capture -> synchronization -> calibration -> triangulation -> 3D kinematics

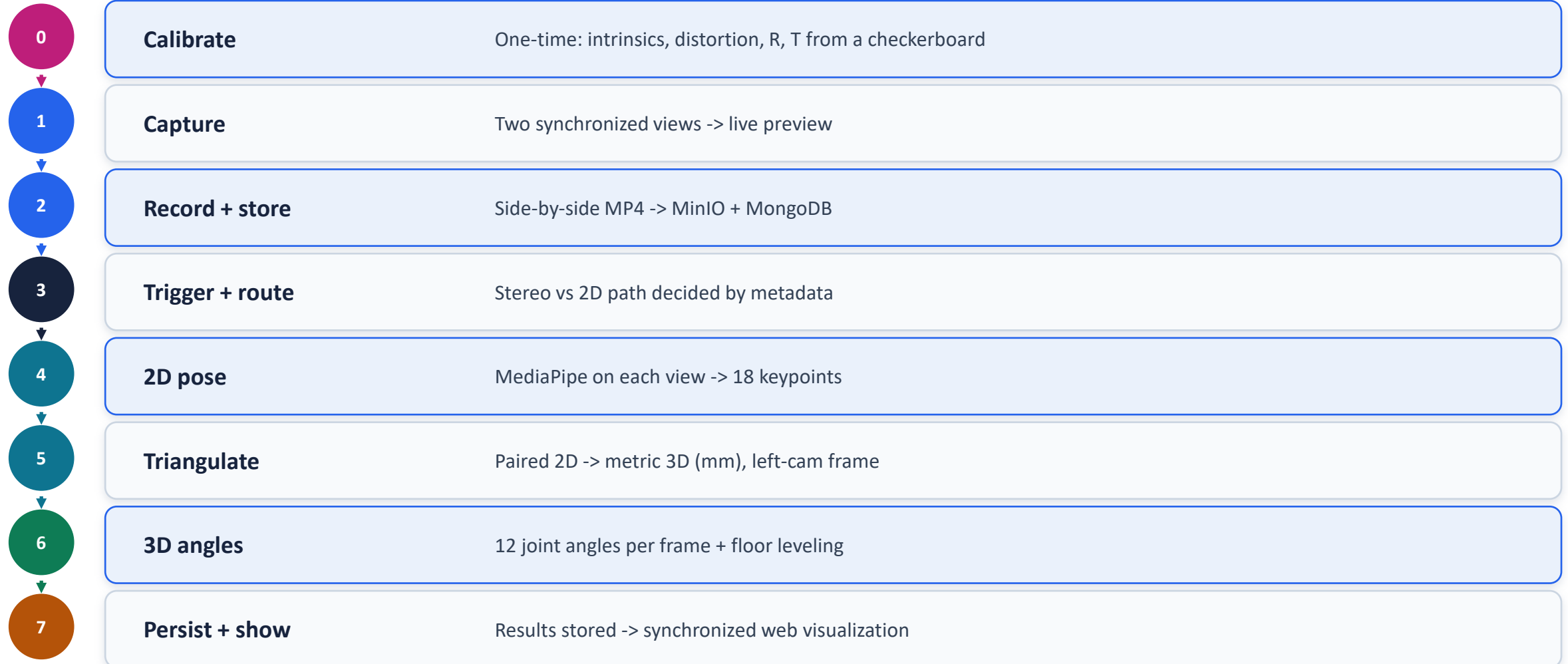
Stereo capture · OpenCV calibration · MediaPipe pose · Metric triangulation



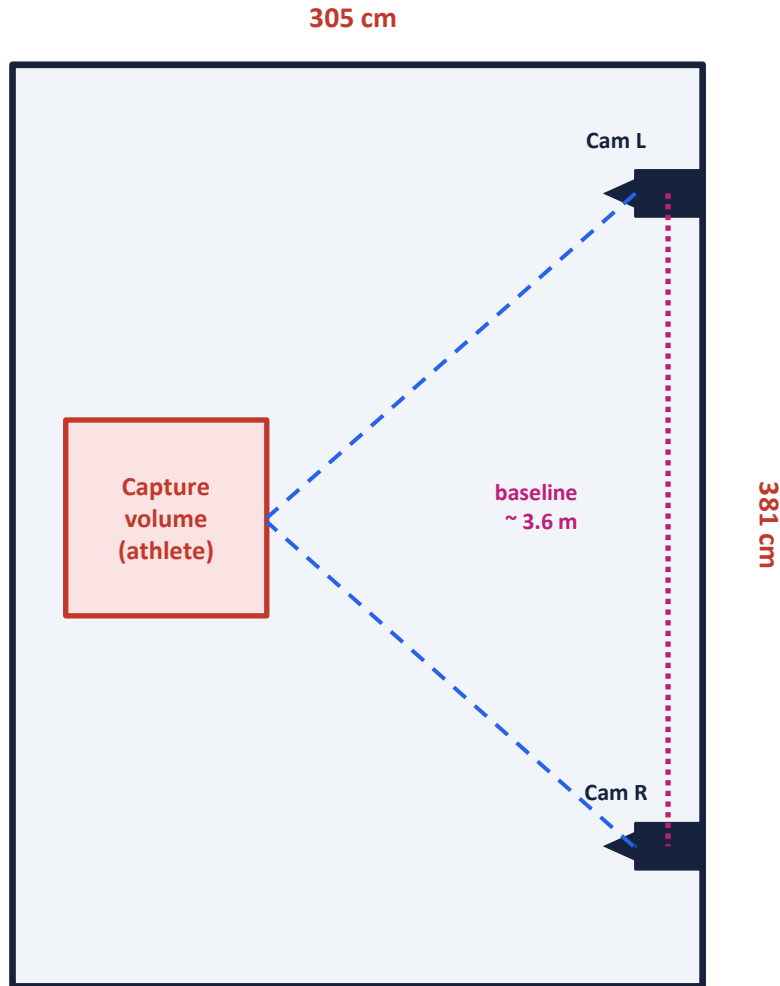
System & Setup

Camera rig, capture & synchronization

End-to-end pipeline



Physical camera rig (top-down)



Rig parameters

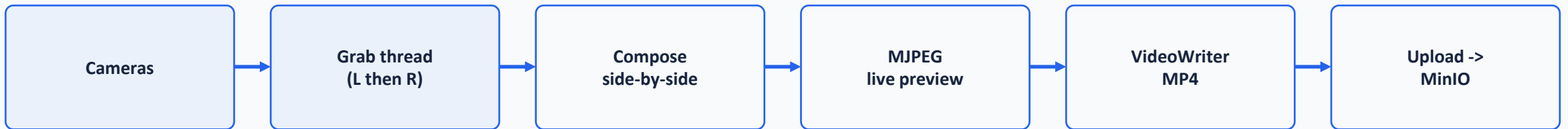
- Two cameras fixed on the same wall, ~3.6 m apart (wide baseline).
- Mounted ~2 m high, both tilted down toward the capture volume.
- Convergent (verged) setup - optical axes meet in the shared volume, not parallel.
- Athlete performs inside the overlapping field of view (the red box) - the only region both cameras see well.
- Trade-off: a wide baseline gives better depth precision but makes calibration & correspondence harder.
- Consequence: the 3D is reconstructed in the LEFT camera's coordinate frame; a floor-leveling step re-orientes it upright.

Capture modes & signal path

USB mode

- Two USB webcams via OpenCV + DirectShow
- MJPG @ 1280x720, buffer size 1
- Fails if a cam is held by OBS/Zoom
- Config: CAMERA_LEFT/RIGHT_INDEX

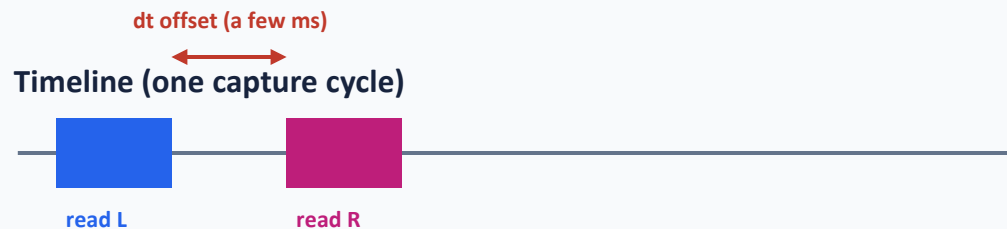
Signal path



Capture & synchronization

How a synchronized pair is formed

- One background thread reads both cameras sequentially: read(L) then read(R).
- Buffer size = 1 to keep only the freshest frame (low latency).
- Last-good frame from each side is cached; newest (L, R, timestamp) is published.
- Live preview served as an MJPEG multipart stream.
- Calibration capture uses the tighter grab()+grab() -> retrieve() pattern.

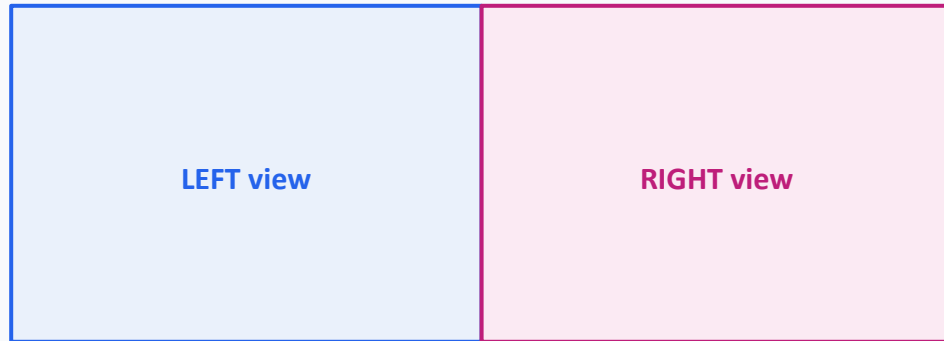


! Software sync - the key limitation

- Not hardware-genlocked. The L/R frames are a few ms-tens of ms apart, and the offset varies under load.
- **Triangulation error $\sim \text{motion_speed} \times \text{dt}$. Worst on fast limbs; negligible on slow/still motion.**
- Example: 20 ms offset, wrist at 2 m/s -> ~4 cm apparent displacement between views.
- Recording uses read()/read() - looser than the grab()/retrieve() used in calibration (an easy internal fix).
- Proper fixes: hardware genlock / external trigger, or timestamp each stream and interpolate to a common clock.

Recording & storage

Side-by-side composite frame (one file)



cv2.hconcat -> one MP4

Frame pacing

- Writer paced to the wall clock: $\text{expected} = \text{elapsed} \times \text{fps}$.
- Duplicates the latest frame to catch up so duration is correct.
- Caveat: fps is nominal - under-delivery -> repeated frames (zero-velocity artifacts).

What gets stored, and where

MinIO (object store)

- Raw recording, then the processed overlay video.
- Unique UUID object names; retrieved via 1-hour presigned URLs.

MongoDB - videos document

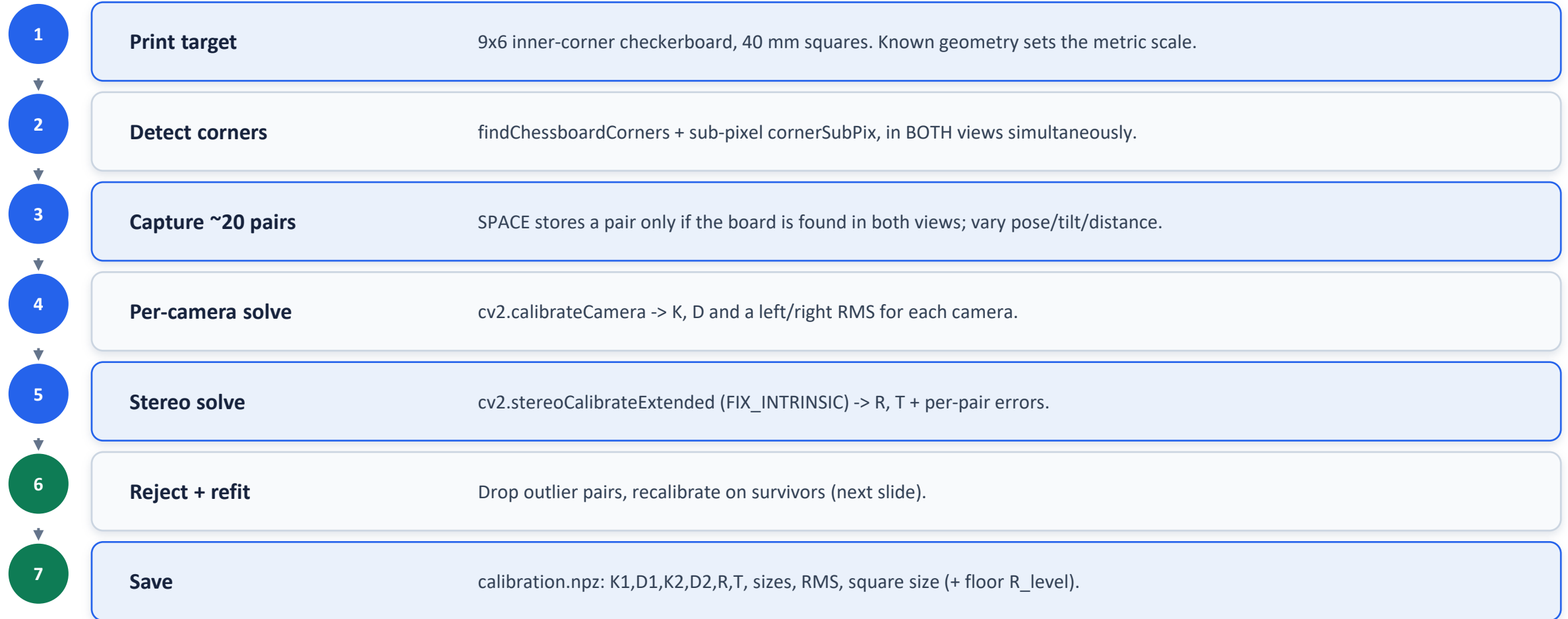
- `user_id`, `bucket`, `object_name`, `filename`, `size`, `uploaded_at`.
- Stereo tags: `source='stereo_camera'`, `layout='side_by_side'`, `fps`, `frames`.
- These tags later route the clip to the 3D pipeline.

Keywords: composite frame - wall-clock pacing - object storage - presigned URL - metadata tagging

Calibration

Teaching the system the geometry of the two cameras

Calibration procedure (offline, one-time)



How bad image pairs are removed

1

Gate 1 - at capture (prevention)

- A pair is stored only if the checkerboard is detected in BOTH views on a fresh synced grab.
- If the board isn't cleanly visible in both -> the pair is never added.

2

Gate 2 - at calibration (outliers)

- stereoCalibrateExtended returns a per-pair reprojection error.
- **threshold = max(1.0 px, 1.5 x median error).**
- Pairs above threshold are dropped; recalibrate on the rest - only if ≥ 8 good pairs remain.

Reprojection error (the metric)

- Project the reconstructed 3D board points back into the image.
- Measure pixel distance to the actually-detected corners.
- **RMS of those distances summarizes the fit - lower is better; target < 1.0 px.**

Honest caveat

- The median x 1.5 rule is a practical filter, not a formal robust estimator (MAD/z-score would be stronger).
- Currently evaluated on the same pairs used to fit - no held-out set yet.

Quality metrics & the wide-baseline challenge

< 1.0 px

Target stereo RMS

~4.2 px

Current stereo RMS (to fix)

3.6 m

Real baseline (measured)

+/-15%

Baseline sanity-check band

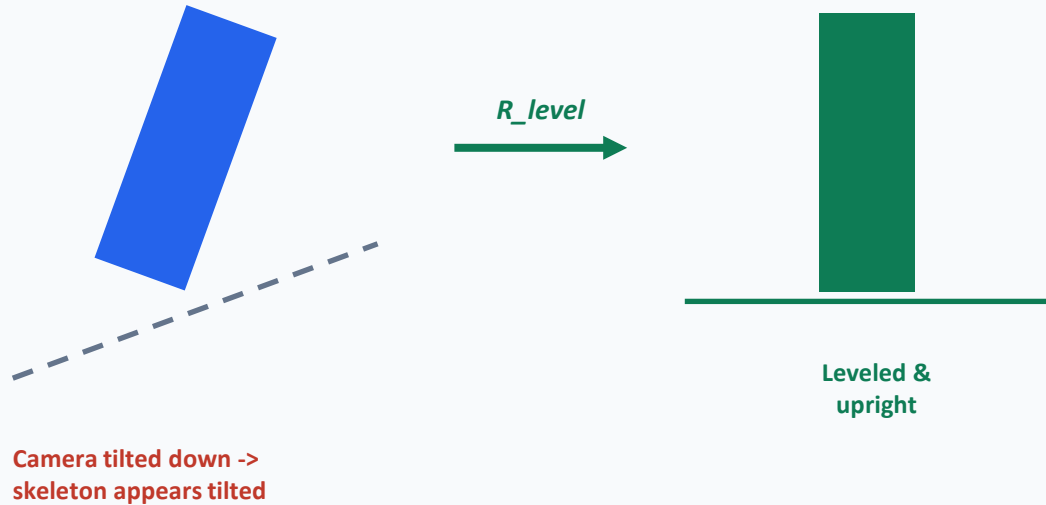
Why our RMS is high right now

- 3.6 m baseline calibrated with a small 40 cm board -> few pixels per corner.
- Steep convergence angle -> the board is foreshortened in at least one view.
- **Per-camera RMS was fine (1.6 / 0.27 px) but stereo RMS was ~4.2 -> a synchronization / correspondence issue, not a lens issue.**
- A per-pair median of 5.6 px meant most pairs were bad, not just outliers.

Fixes in progress

- Larger / higher-resolution calibration target.
- Hold the board perfectly still; keep both cameras rigid.
- Added baseline sanity check: |T| vs measured 3.6 m, warns if >15% off.
- Added floor-leveling to fix orientation independently.
- **Realistic goal for this rig: ~1-1.5 px.**

Floor leveling - standing the skeleton upright



How it works

- Lay the checkerboard flat on the floor; capture it in both cameras (press 'f').
- Triangulate its corners -> fit a plane -> the plane normal is 'up'.
- Compute a rotation R_{level} that maps that normal to the vertical axis.
- Store R_{level} in calibration.npz; apply it to every 3D frame at analysis time.
- **Joint ANGLES are rotation-invariant, so leveling changes only the display orientation - never the measured numbers.**

The 3D Pipeline

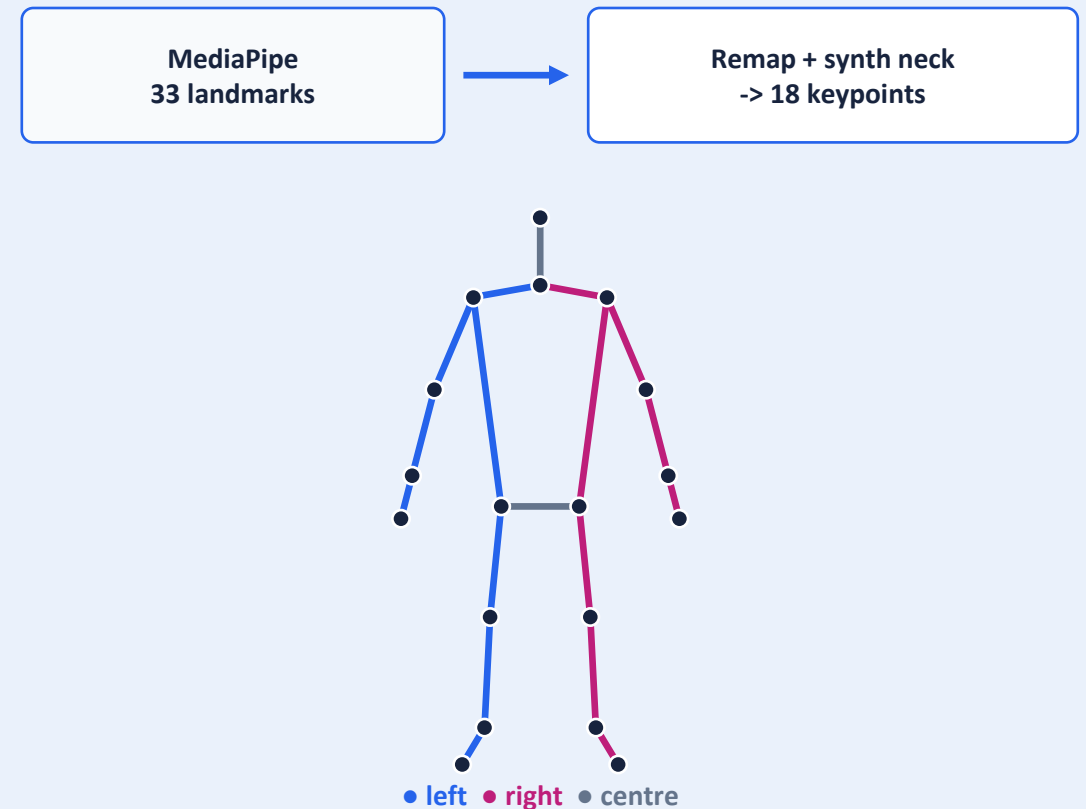
From pixels to metric 3D joint angles, then to the screen

2D pose estimation (per view)

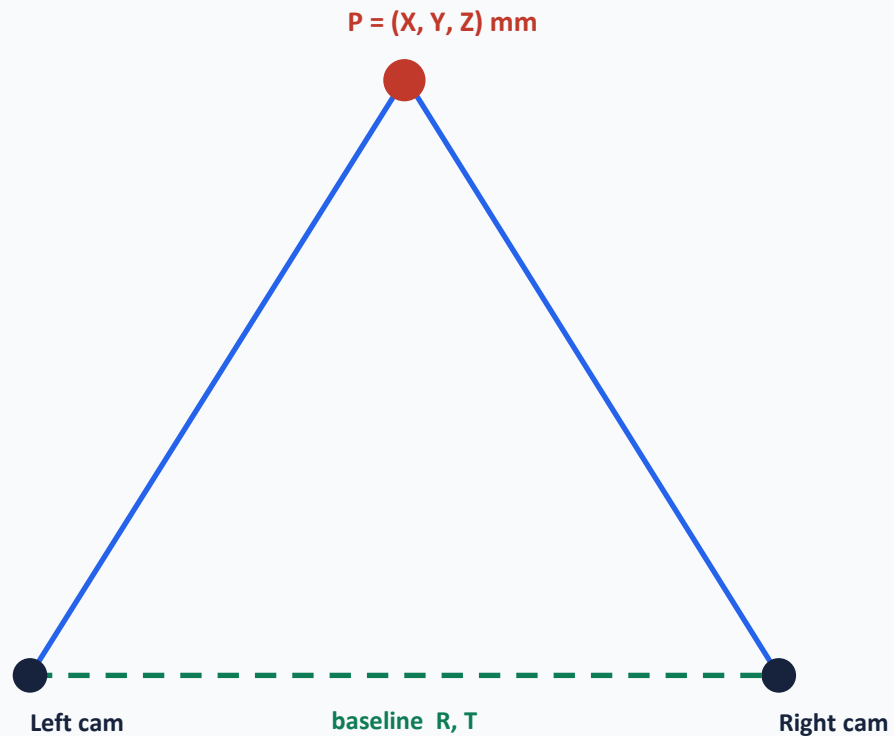
MediaPipe Pose Landmarker

- Tasks API, VIDEO running mode, model file pose_landmarker.task.
- Per frame: BGR->RGB -> detect_for_video with a 30 fps-locked timestamp.
- Returns 33 landmarks; we remap to a custom 18-point skeleton and synthesize a NECK = midpoint of the two shoulders.
- Keypoints stored NORMALIZED [x, y, presence] in 0-1.
- **For stereo, this runs twice - left half and right half - with temporal healing OFF.**
- We do NOT use MediaPipe's z (monocular, relative). Real depth comes from triangulation.

33 landmarks -> 18-point skeleton



Paired 2D -> metric 3D



Two rays, one intersection -> depth

Method (standard, robust)

- Convert normalized keypoints -> pixels using each view's stored size.
- `cv2.undistortPoints` - removes distortion + intrinsics -> normalized camera rays.
- **Projection matrices:** $P1 = [I \mid 0]$, $P2 = [R \mid T]$.
- `cv2.triangulatePoints` (linear DLT) -> homogeneous -> divide -> 3D in the LEFT-camera frame, in mm.
- A joint below 0.5 confidence in EITHER view -> NaN (gap), never a wrong point.

Limits: linear DLT (algebraic error), no per-joint outlier rejection, no bundle adjustment, no epipolar correspondence check.

3D joint angles

Computation

- For a joint at B with neighbours A and C: form vectors A-B and C-B.
- Angle = arccos of their normalized dot product - a true 3D angle.
- 12 angles per frame: knees, hips, elbows, wrists, shoulders, ankles (L & R).
- A missing (NaN) endpoint -> that angle is simply skipped for the frame.

$$\text{theta} = \arccos((\mathbf{A-B}) \cdot (\mathbf{C-B}) / |\mathbf{A-B}| |\mathbf{C-B}|)$$

in 3D - no camera-facing heuristics needed

Why this output is robust

- **Angles are INVARIANT to scale** - a wrong square size mis-sizes positions but not angles.
- **Angles are INVARIANT to global rotation** - camera tilt / leveling doesn't change them.
- So many calibration errors hurt the 3D picture more than the numbers we actually report.
- The 2D path additionally uses a facing-direction normalization + anatomical convention; the 3D path needs none of that.

Storing the results

On the process-video request

- Download clip from MinIO to a temp file.
- Route: layout=='side_by_side' (or source=='stereo_camera') AND calibration.npz exists -> 3D path; else 2D.
- Run pose + triangulation; render the left-view overlay video.
- Re-upload the overlay to MinIO as processed_object_name.
- Runs synchronously inside the HTTP request (a scalability limit).

pose_analysis document

- video_id, user_id, status, created_at.
- result.frames - per-frame 2D keypoints + 2D angles (drives the graph).
- result.frames_3d - triangulated 3D points + true 3D angles (drives the 3D viewer).
- result.mode = 'stereo_3d', units = 'mm', fps, total_frames.
- One document per video; upsert on re-processing.

Frontend & time synchronization

Analysis page - 3 synced panels

Overlay video
(2D skeleton)

Rotatable 3D
skeleton viewer

Joint-angle graph (ECharts) - angle vs time

Event-driven time sync

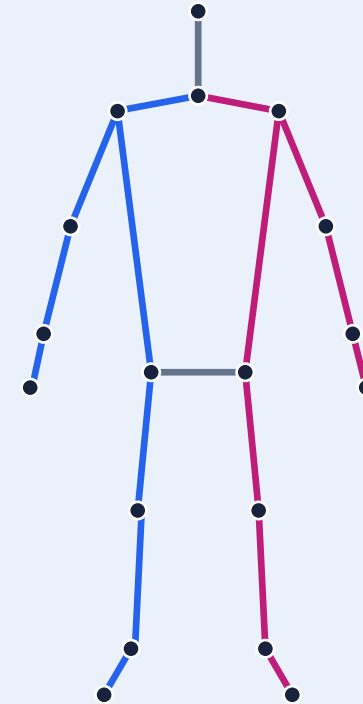
- On each animation frame the video dispatches a 'sync-time' window event with currentTime.
- The graph moves a vertical marker line to that time (throttled ~20 fps).
- The 3D viewer jumps to frame = floor(time x fps).
- 'analysis-complete' event invalidates the TanStack Query cache to refetch results.

Keywords: SPA - custom DOM events - requestAnimationFrame - throttling - query invalidation - presigned playback URL

The 3D skeleton viewer

Rendering

- Pure HTML Canvas - a hand-written 3D->2D projection (no Three.js).
- Manual yaw / pitch / zoom via mouse drag + wheel.
- Auto-centers and scales to the reconstructed point cloud.
- Draws bones (left/right/centre coloured), joints, and a floor grid.
- Live HUD lists the current frame's 3D joint angles.



Reads frames_3d for the selected video; rotates with R_level so the athlete stands upright.

End-to-end data flow



Offline, once

- Stereo calibration -> calibration.npz (K, D, R, T).
- Floor leveling -> R_level.
- Reused by every subsequent recording.

Per clip, on demand

- Triggered by the 'Extract Kinematics' button.
- Everything from split -> 3D -> store runs server-side.
- Results cached; UI refetches on completion.

Known limitations

Synchronization

Software-level sequential reads; no genlock. Recording sync looser than calibration.

Triangulation

Linear DLT, no per-joint outlier rejection, no bundle adjustment.

Calibration

~4.2 px RMS now; fit-set evaluation; median x 1.5 outlier rule is heuristic.

Confidence

Uses 'presence', not 'visibility'; no cross-view occlusion fallback.

Correspondence

No epipolar check before triangulation - a MediaPipe L/R label flip would mis-pair joints.

Temporal filtering

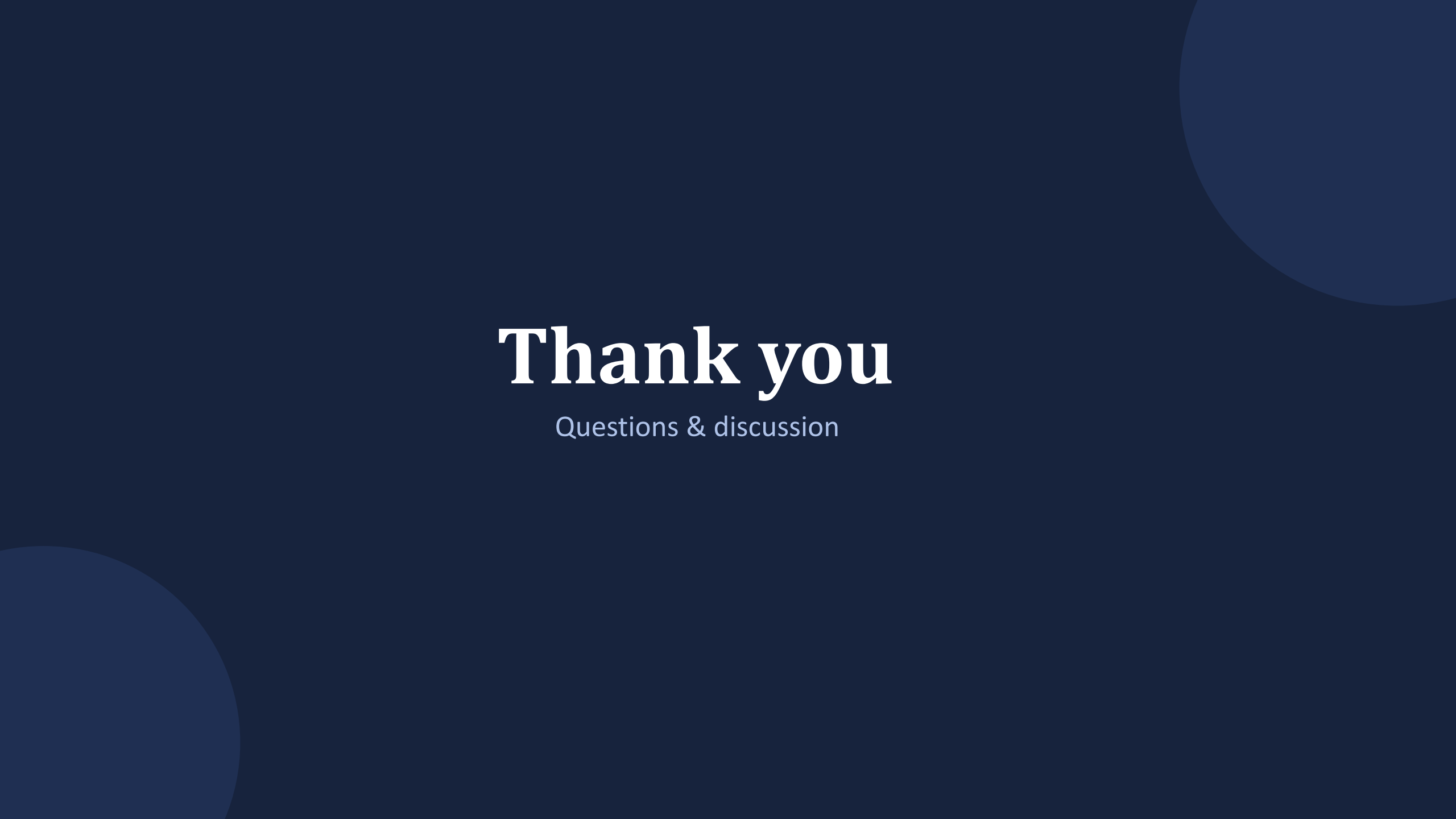
3D path runs raw (healing off) - no Kalman / One-Euro smoothing.

Frame rate

fps hardcoded to 30; nominal fps via frame duplication.

Scope

Single person; synchronous processing; accuracy not yet quantified.



Thank you

Questions & discussion